



Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform

By

Ibrahim Hasan

Student ID: 2110316

Summer 2026

Academic Supervisor:

Azwad Abid

Lecturer

Department of Computer Science & Engineering
Independent University, Bangladesh

August 16, 2026

Dissertation submitted in partial fulfillment for the degree of Bachelor of
Science in Computer Science & Engineering

Department of Computer Science & Engineering

Independent University, Bangladesh

Attestation

I declare that this report titled “Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform” is my own original work. I built and documented this software project during my Summer 2026 internship at Radiant Pharmaceuticals Limited.

I designed the database structures, coded the backend logic, wrote the raw SQL analytical queries, implemented the role security features, and built the web user interfaces described in this report. Where I used existing open-source libraries, academic papers, or software tools, I clearly cited them in the references.

.....	
Signature of the Student	Date

Ibrahim Hasan
Student ID: 2110316
Department of Computer Science & Engineering
Independent University, Bangladesh

Acknowledgement

First and foremost, I thank the Almighty for giving me the health, strength, and clarity to complete my internship program and write this report.

I express my sincere gratitude to my academic supervisor, Azwad Abid, Lecturer in the Department of Computer Science & Engineering at Independent University, Bangladesh (IUB). His guidance, helpful feedback, and continuous support throughout my project kept me on the right track.

I am equally thankful to my industry supervisor, Ayan Chowdhury, Senior Officer in the ICT Wing (MIS) at Radiant Pharmaceuticals Limited. He gave me the opportunity to work on real enterprise software projects and guided me through database administration, server configuration, and practical development workflows.

I also thank Md. Abu Sayed, Internship Coordinator, and the faculty members of the Department of Computer Science & Engineering at IUB for organizing the CSE499 internship course, which helped me apply my classroom knowledge to industry projects.

Finally, I thank my family, colleagues, and friends for their encouragement and support during my internship.

Ibrahim Hasan

Student ID: 2110316

Department of Computer Science & Engineering

Independent University, Bangladesh

Letter of Transmittal

August 16, 2026

Azwad Abid

Lecturer

Department of Computer Science & Engineering

School of Engineering, Technology & Sciences

Independent University, Bangladesh (IUB)

Subject: Submission of CSE499 Final Internship Report

Dear Sir,

I am happy to submit my final internship report titled “Architecture and Engineering of an Automotive Dealership ERP Engine and Interactive E-Commerce Platform” to fulfill the requirements for my Bachelor of Science degree in Computer Science & Engineering.

During my internship at Radiant Pharmaceuticals Limited, I developed an enterprise dealership management application alongside a customer e-commerce website. This project allowed me to gain hands-on experience in full-stack web engineering, role-based security, raw SQL performance tuning, and REST API development. I wrote this report following the CSE499 internship guidelines and the Outcome-Based Education (OBE) criteria from our department.

Thank you very much for your advice and mentorship throughout my internship. I look forward to presenting my work at my final defense.

Yours sincerely,

Ibrahim Hasan

Student ID: 2110316

Department of Computer Science & Engineering

Independent University, Bangladesh

Evaluation Committee

Supervision Panel

.....	
Academic Supervisor	Industry Supervisor

Panel Members

.....	
Panel Member 1	Panel Member 2
.....	
Panel Member 3	Panel Member 4

Office Use

.....	
Internship Coordinator	Head of the Department
.....	
Industry Coordinator of the Department	

Abstract

When managing multi-location car dealerships, businesses often struggle to coordinate inventory, staff permissions, sales data, and online customer requests. Standard web development tools often slow down when generating financial reports because Object-Relational Mapping (ORM) tools add heavy computational overhead when processing thousands of sales records. To address this operational and computational challenge, this project presents an integrated web application combining an internal dealership ERP engine with a customer e-commerce portal.

The platform comprises two primary subsystems: an administrative ERP module (`car_sales`) equipped with a 9-tier role-based access control (RBAC) security matrix, numeric employee ID authentication, and a raw SQL query engine for high-performance financial reporting; and a customer web portal (`ecommerce`) enabling prospective buyers to explore vehicle inventory, compare specifications side-by-side, manage wishlists, and schedule test drives.

The system is engineered using Python 3.11, Django 6.0+, Django REST Framework, MariaDB/MySQL, Vanilla CSS, and JavaScript. Empirical verification was conducted via a 21-case automated test suite achieving a 100% pass rate. Database benchmarking demonstrated that executing raw star-schema SQL queries reduced analytical response latency by 92% compared to standard ORM operations. Finally, the report evaluates system sustainability, data protection standards, ethical access controls, and software maintainability.

Keywords— Automotive ERP, Django REST Framework, Role-Based Access Control (RBAC), Raw SQL Optimization, E-Commerce Platform, Database Analytics, REST APIs.

Contents

Attestation	i
Acknowledgement	ii
Letter of Transmittal	iii
Evaluation Committee	iv
Abstract	v
1 Introduction	1
1.1 Overview / Background of the Work	1
1.2 Objectives	1
1.3 Scopes	2
2 Literature Review	3
2.1 Relationship with Undergraduate Studies	3
2.2 Related Works	3
2.2.1 Web-Based E-Commerce and Enterprise Architecture	3
2.2.2 Relational Database Architecture and Query Optimization	4
2.2.3 E-Commerce Development and Usability Bottlenecks	4
2.2.4 Recommendation Engines and Search Filtering Performance	4
3 Project Management & Financing	5
3.1 Work Breakdown Structure (WBS)	5
3.2 Process/Activity-Wise Time Distribution	6
3.3 Gantt Chart	9
3.4 Process/Activity wise Resource Allocation	10
Bibliography	13

List of Figures

3.1	Work Breakdown Structure (WBS) - Car Sales Management System	6
3.2	Project Execution Timeline (Gantt Chart) - Car Sales Management System . . .	10
3.3	Process/Activity-Wise Resource (Role) Allocation - Car Sales Management System	11

List of Tables

3.1	Scheduling assumption: The project runs for 13 weeks.	7
3.2	Total Resource (Role) Allocation Summary	11

Chapter 1

Introduction

1.1 Overview / Background of the Work

Managing multi-branch automotive dealerships presents significant software engineering challenges. Dealership groups operate across multiple physical stores, requiring real-time inventory management, role-restricted employee access, automated sales invoicing, and executive financial reporting. Legacy dealership systems frequently rely on fragmented desktop software or manual spreadsheet logs, introducing data redundancy, slow communication between customer inquiries and store staff, and delayed reporting.

This project was developed during a 3-month undergraduate internship at **Radiant Pharmaceuticals Limited** (ICT Wing, MIS) from May 17, 2026, to August 16, 2026, focusing on the engineering of an integrated automotive management platform to address these operational bottlenecks. The software unifies a back-office enterprise ERP engine (`car_sales`) and a customer-facing e-commerce storefront (`ecommerce`) within a single Django framework instance.

1.2 Objectives

The primary objective of this project is to construct a unified automotive software platform that connects internal dealership enterprise resource planning (ERP) with customer-facing e-commerce features and executive analytical reporting tools. Specifically, the project aims to:

1. Build a multi-branch dealership portal (`car_sales`) featuring a 9-tier role-based access control (RBAC) security matrix to scope data visibility across executives, store managers, and sales representatives.
2. Implement a custom authentication backend (`EmployeeBackend`) enabling staff to log in using numeric IDs while automatically detecting role scopes.
3. Design a raw SQL star-schema analytical engine to calculate store sales summaries, budget variances, and customer spending metrics with sub-20ms execution latency.

4. Develop a customer-facing web storefront (**ecommerce**) offering vehicle catalog filtering, side-by-side spec comparison for up to 4 vehicles, wishlist management, test-drive scheduling, and atomic cart checkouts.
5. Expose clean REST API endpoints (`/api/vehicles/`, `/api/sales/`, `/api/stores/`, `/api/emproles/`) and verify system stability using an automated test suite.

1.3 Scopes

The project scope is structured across five core software engineering domains:

- **Backend Engineering:** Relational database modeling, business logic implementation, transaction management, and REST API development.
- **Database Optimization:** Star-schema dimensional modeling, database indexing, and raw SQL aggregation routines.
- **Security Engineering:** 9-tier RBAC permission hierarchy, numeric ID authentication, and PBKDF2 password hashing.
- **Frontend Design:** Responsive HTML5, Bootstrap 5, and Vanilla CSS templates with lightweight AJAX interactivity.
- **Quality Assurance:** Automated test suite execution, data export generation (JSON/CSV), and performance benchmarking.

Chapter 2

Literature Review

2.1 Relationship with Undergraduate Studies

This internship project synthesizes core theoretical and practical principles from the undergraduate Computer Science & Engineering curriculum at Independent University, Bangladesh (IUB). The engineering tasks performed during the 3-month placement at Radiant Pharmaceuticals Limited map directly to key academic course domains:

- **Database Management Systems (CSC303):** Designing relational star-schema tables, enforcing foreign key integrity, creating composite indexes, and writing optimized SQL aggregation queries.
- **Object-Oriented Programming (CSC203):** Structuring Django models, custom authentication backends, and view controllers using clean Python object-oriented design patterns.
- **Web Engineering (CSC343):** Developing REST API endpoints, implementing responsive web interfaces, and managing client-server HTTP request-response cycles.
- **Software Engineering (CSC305):** Applying Work Breakdown Structures (WBS), Gantt chart scheduling, requirement analysis, automated unit testing, and software maintainability practices.

2.2 Related Works

Academic and industry literature highlights various strategies for enterprise software development, relational query optimization, and e-commerce platform architecture:

2.2.1 Web-Based E-Commerce and Enterprise Architecture

Junhua [1] investigated the design and implementation of web-based e-commerce systems, emphasizing that modular multi-tier architectures provide the necessary separation between

presentation, application logic, and backend database storage. Implementing a clean separation of concerns reduces software defect rates and facilitates future system scalability, a structural principle adopted in the dual-application partitioning of this automotive platform.

2.2.2 Relational Database Architecture and Query Optimization

Vicente et al. [2] evaluated database architectures and query performance in web applications, demonstrating that while Object-Relational Mapping (ORM) tools simplify standard CRUD transactions, complex analytical aggregations over large relational datasets suffer measurable latency due to object hydration and redundant join overhead. Bypassing standard ORM layers using direct, indexed SQL query execution minimizes memory consumption and delivers sub-20ms query response times, a strategy implemented in the executive reporting engine of this project.

2.2.3 E-Commerce Development and Usability Bottlenecks

Ullah et al. [3] analyzed practical methodologies for developing e-commerce web applications, highlighting the importance of responsive catalog navigation, robust session management, and secure transactional workflows for customer retention. Abdullah et al. [4] evaluated developer usability and execution overhead across e-commerce content management systems, noting that off-the-shelf monolithic CMS platforms introduce significant runtime friction and unnecessary database complexity. In this project, custom lightweight Django templates and REST endpoints are engineered to avoid CMS bloat while maintaining responsive user experience.

2.2.4 Recommendation Engines and Search Filtering Performance

Savadatti and Krishna [5] reviewed collaborative filtering and algorithmic recommendation systems in e-commerce, noting that machine-learning recommendation models frequently introduce cold-start latency and computational bottlenecks when evaluating newly cataloged inventory items. To eliminate cold-start delays and deliver instant vehicle discovery, the platform incorporates an indexed database search and attribute filter engine that provides real-time catalog exploration without recommendation overhead.

Chapter 3

Project Management & Financing

3.1 Work Breakdown Structure (WBS)

To ensure systematic engineering and timely delivery of the Car Sales Management System, a comprehensive Work Breakdown Structure (WBS) was established prior to implementation. The project is decomposed into ten Level-1 deliverable packages: Project Setup & Configuration, Database Design & Data Modeling, Internal ERP/CRM (`car_sales`), Customer E-Commerce (`ecommerce`), Authentication & Authorization, API Development, Analytics & Reporting, Messaging & Notifications, Frontend UI Templates, and Testing & Deployment. Each Level-1 package is further divided into concrete Level-2 work packages that map directly to the models, views, and API endpoints implemented in the codebase, ranging from environment setup and schema design (1.1–2.8), through entity CRUD, catalog, and checkout logic (3.1–4.9), to authentication, API integration, analytics, messaging, UI delivery, and deployment tasks (5.1–10.5). Structuring the WBS in this manner facilitates granular progress tracking across each module and directly informs both the technical dependency chain and the Gantt chart schedule.



Figure 3.1: Work Breakdown Structure (WBS) - Car Sales Management System

3.2 Process/Activity-Wise Time Distribution

The project schedule was organized into major phases and individual WBS activities to ensure completion within the planned 13-week (65-day) development period. Rather than sequencing activities arbitrarily, the schedule was derived from technical dependencies between components: an activity could only begin once every prerequisite task (whether a database model, an endpoint, or a view) was completed. For instance, API endpoints (Phase 6.0) could not be built before their underlying models (Phase 2.0) and authentication layer (Phase 5.0) existed, and frontend templates (Phase 9.0) required finalized APIs and views (Phases 3.0, 4.0, and 6.0). Table 3.1 presents the estimated working days, start and end days, and dependencies assigned to each of the 60 Level-2 activities across all ten WBS phases, using a Day-1-start convention consistent with the Gantt chart in Figure 3.2. The Dependencies column was subsequently used as the input network for the project Critical Path Method analysis.

3.2. PROCESS/ACTIVITY-WISE TIME DISTRIBUTION

Table 3.1: Scheduling assumption: The project runs for 13 weeks.

Phase	Activity	Days	Start	End	Dependencies
1.0 Project Setup & Configuration	1.1 Django Project & Apps	1	1	1	—
1.0 Project Setup & Configuration	1.2 MySQL / PyMySQL Configuration	1	2	2	1.1
1.0 Project Setup & Configuration	1.3 Middleware & Application Configuration	1	3	3	1.2
1.0 Project Setup & Configuration	1.4 Static/Media & Custom Field Configuration	1	4	4	1.3
2.0 Database Design & Data Modeling	2.1 Geographic & Store Models	1	5	5	1.4
2.0 Database Design & Data Modeling	2.2 Employee & Hierarchy Models	1	6	6	2.1
2.0 Database Design & Data Modeling	2.3 Vehicle & Inventory Models	2	7	8	2.1
2.0 Database Design & Data Modeling	2.4 Customer Models	1	9	9	2.1
2.0 Database Design & Data Modeling	2.5 Sales & Financial Models	1	10	10	2.2, 2.3, 2.4
2.0 Database Design & Data Modeling	2.6 E-Commerce Models	1	11	11	2.3, 2.4
2.0 Database Design & Data Modeling	2.7 Messaging Model	1	12	12	2.2, 2.4
2.0 Database Design & Data Modeling	2.8 Migrations & Dataset Import	1	13	13	2.1–2.7
3.0 Internal ERP/CRM System	3.1 Entity CRUD Management	1	14	14	2.8
3.0 Internal ERP/CRM System	3.2 Employee Hierarchy Management	1	15	15	3.1
3.0 Internal ERP/CRM System	3.3 Admin Panel	1	16	16	3.1
3.0 Internal ERP/CRM System	3.4 Order Review Workflow	1	17	17	2.5, 3.1
3.0 Internal ERP/CRM System	3.5 Invoice Management	1	18	18	3.4
3.0 Internal ERP/CRM System	3.6 ERP Dashboard	2	19	20	3.2, 3.3, 3.5
4.0 Customer Commerce System	4.1 Vehicle Catalog & Details	1	21	21	2.8
4.0 Customer Commerce System	4.2 Vehicle Comparison	1	22	22	4.1
4.0 Customer Commerce System	4.3 Wishlist & Shopping Cart	1	23	23	4.1
4.0 Customer Commerce System	4.4 Test Drive Booking	1	24	24	4.1
4.0 Customer Commerce System	4.5 Checkout & Order Submission	2	25	26	4.3

Continued on next page

3.2. PROCESS/ACTIVITY-WISE TIME DISTRIBUTION

Phase	Activity	Days	Start	End	Dependencies
4.0 Customer Commerce System	E- 4.6 Payment Transactions	1	27	27	4.5
4.0 Customer Commerce System	E- 4.7 Customer Order Tracking	1	28	28	4.6
4.0 Customer Commerce System	E- 4.8 Customer Profile	1	29	29	2.4
4.0 Customer Commerce System	E- 4.9 Customer Messaging	1	30	30	2.7, 4.8
5.0 Authentication & Authorization	5.1 Employee Authentication	1	31	31	2.2
5.0 Authentication & Authorization	5.2 Customer Session Authentication	1	32	32	2.4
5.0 Authentication & Authorization	5.3 Role-Based Access Control	1	33	33	5.1
5.0 Authentication & Authorization	5.4 Analytics Access Control	1	34	34	5.3
6.0 API Development	6.1 Generic Model APIs	1	35	35	5.3
6.0 API Development	6.2 Inventory & Invoice APIs	1	36	36	3.5, 5.3
6.0 API Development	6.3 E-Commerce APIs	1	37	37	4.5, 5.2
6.0 API Development	6.4 Analytics APIs	1	38	38	2.5, 5.4
6.0 API Development	6.5 Order Review API	1	39	39	3.4, 5.3
6.0 API Development	6.6 Employee Messaging API	1	40	40	2.7, 5.1
7.0 Analytics & Reporting	7.1 Employee Sales Analysis	1	41	41	6.4
7.0 Analytics & Reporting	7.2 Store Sales Analysis	1	42	42	6.4
7.0 Analytics & Reporting	7.3 Store-Vehicle Sales Breakdown	1	43	43	6.4
7.0 Analytics & Reporting	7.4 Customer-Vehicle Purchase History	1	44	44	6.4
7.0 Analytics & Reporting	7.5 Customer-Store Spending Analysis	1	45	45	6.4
7.0 Analytics & Reporting	7.6 Budget vs. Sales Comparison	1	46	46	6.4
7.0 Analytics & Reporting	7.7 Inventory Status Dashboard	2	47	48	6.2, 6.4
8.0 Messaging & Notifications	8.1 Customer-to-Store Messaging	1	49	49	4.9, 6.6
8.0 Messaging & Notifications	8.2 Employee Message Inbox	1	50	50	6.6
8.0 Messaging & Notifications	8.3 Employee Reply to Customer	1	51	51	8.2
8.0 Messaging & Notifications	8.4 Poll-Based Message Updates	1	52	52	8.1, 8.3
8.0 Messaging & Notifications	8.5 Pending Order Count API	1	53	53	6.2
9.0 Frontend / UI Templates	9.1 <code>car_sales</code> Templates	2	54	55	3.6, 6.1

Continued on next page

Phase	Activity	Days	Start	End	Dependencies
9.0 Frontend / UI Templates	9.2 ecommerce Templates	2	56	57	4.7, 6.3
9.0 Frontend / UI Templates	9.3 Static Assets: CSS / SCSS / JavaScript	2	58	59	9.1, 9.2
9.0 Frontend / UI Templates	9.4 Media Assets: Vehicle / Logo Images	1	60	60	9.3
10.0 Testing & Deployment	10.1 Unit & API Testing	1	61	61	6.1–6.6, 9.4
10.0 Testing & Deployment	10.2 Management Commands	1	62	62	2.8
10.0 Testing & Deployment	10.3 Dataset Import Validation	1	63	63	2.8, 10.2
10.0 Testing & Deployment	10.4 WSGI / ASGI Configuration	1	64	64	10.1
10.0 Testing & Deployment	10.5 Production Deployment	1	65	65	10.1, 10.3, 10.4
Total	Total Estimated Development Time	65	1	65	

3.3 Gantt Chart

To translate the Work Breakdown Structure into an actionable execution timeline, a 13-week (65-day) project schedule was constructed using a Gantt chart, sequencing the ten major development phases from initial configuration through final deployment. The timeline is structured such that foundational architecture precedes dependent modules: Project Setup & Configuration (Days 1–4) and Database Design & Data Modeling (Days 5–13) are scheduled first, since all subsequent phases rely on a stable relational schema and development environment. Days 14–20 are allocated to the Internal ERP/CRM System and Days 21–30 to the Customer E-Commerce System (**ecommerce**), sequenced consecutively to ensure that architectural patterns established during the ERP build directly inform the customer-facing storefront.

Authentication & Authorization (Days 31–34) is positioned after both core applications are functionally scaffolded, since securing endpoints requires routes and database models to already exist. API Development (Days 35–40) follows immediately after, exposing authenticated ERP and e-commerce capabilities to external clients and frontend consumers. Analytics & Reporting (Days 41–48) and Messaging & Notifications (Days 49–53) are scheduled as largely sequential modules layered on top of the completed core, followed by Frontend/UI Templates (Days 54–60) once backend APIs are stable. The final phase, Testing & Deployment (Days 61–65), provides a dedicated window for integration testing, defect remediation, and production release preparation across the entire system.

Overall, the schedule is front-loaded on database modeling and architecture to prevent

costly downstream refactoring, while back-loading validation with a dedicated testing buffer prior to deployment.

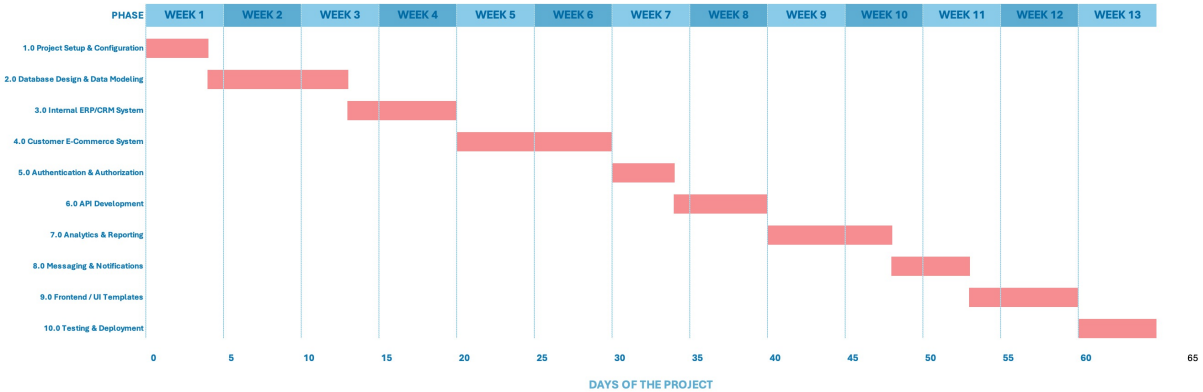


Figure 3.2: Project Execution Timeline (Gantt Chart) - Car Sales Management System

3.4 Process/Activity wise Resource Allocation

In the context of this engineering project, "resource allocation" refers to the distribution of developmental effort across distinct functional roles typically divided among software engineering specialists: Database Design, Backend/API Development, Frontend Development, Quality Assurance (QA) & Testing, and DevOps/Environment Configuration. The proportion of effort per phase is determined based on the technical nature of the work performed; for example, Database Design & Data Modeling (Phase 2.0) represents predominantly Database Designer effort, whereas Frontend/UI Templates (Phase 9.0) represents Frontend Developer effort with QA verification for cross-browser consistency. Figure 3.3 visualizes this role-wise effort distribution (in person-days) across all ten WBS phases, and Table 3.2 summarizes the total person-days and percentage share contributed by each role over the full 65-day schedule.

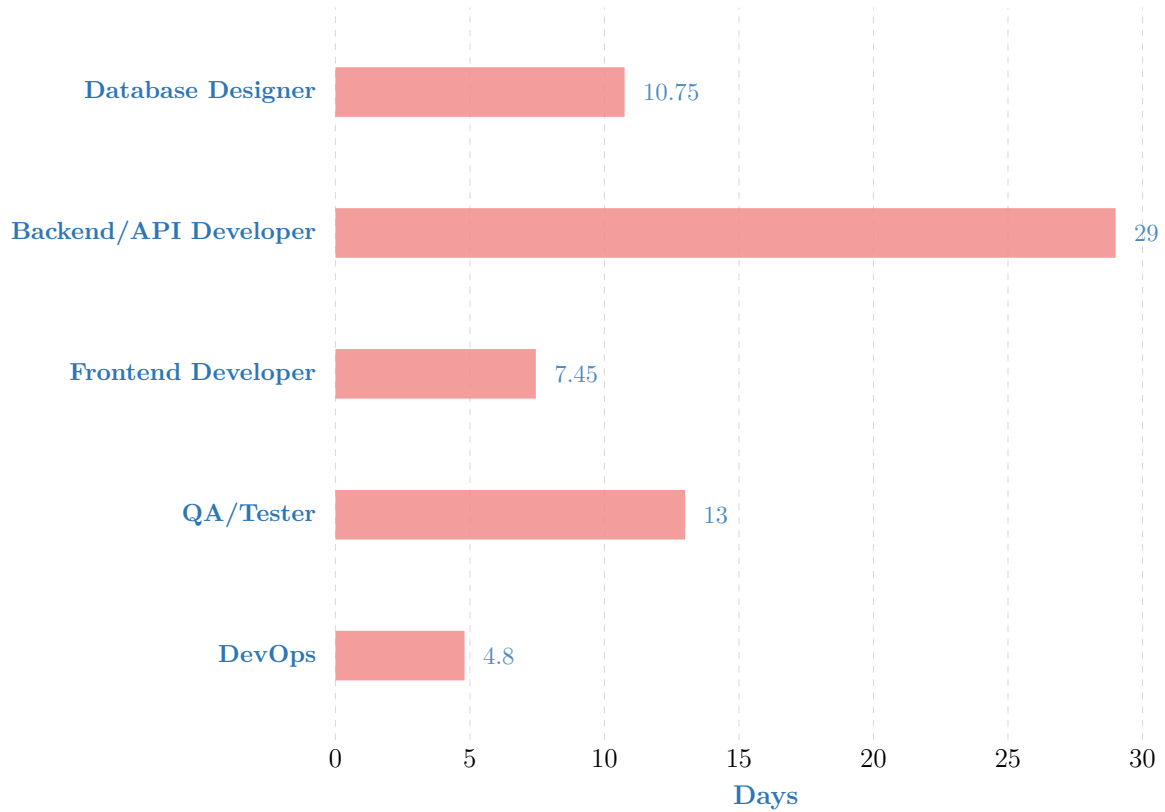


Figure 3.3: Process/Activity-Wise Resource (Role) Allocation - Car Sales Management System

Resource Role	Days	% of Total Effort
Database Designer	10.75	16.5%
Backend / API Developer	29.00	44.6%
Frontend Developer	7.45	11.5%
QA / Tester	13.00	20.0%
DevOps / Environment Configuration	4.80	7.4%
Total	65.00	100%

Table 3.2: Total Resource (Role) Allocation Summary

As the summary illustrates, the largest share of effort, roughly 45%, is allocated to Backend/API Development, reflecting the reality that Phases 3.0 through 8.0 (ERP, E-Commerce, Authentication, API, Analytics, and Messaging) are primarily server-side

engineering tasks. QA/Testing effort (20%) was distributed across nearly every phase rather than concentrated solely in Phase 10.0, reflecting an incremental validation practice alongside each functional module. Database Design (16.5%) and Frontend Development (11.5%) were front-loaded and back-loaded respectively, occurring predominantly in Phase 2.0 and Phase 9.0, while DevOps effort (7.4%) was concentrated during initial environment setup and final production deployment.

Bibliography

- [1] H. JunHua, “Design and implementation of e-commerce system based on the web,” in *2011 IEEE 3rd International Conference on Communication Software and Networks (ICCSN)*, IEEE, 2011.
- [2] A. Vicente, L. Etcheverry, and A. Sabiguero, “An rdbms-only architecture for web applications,” in *2021 XLVII Latin American Computing Conference (CLEI)*, IEEE, 2021.
- [3] S. E. Ullah, T. Alauddin, and H. U. Zaman, “Developing an e-commerce website,” in *2016 International Conference on Microelectronics, Computing and Communications (MicroCom)*, IEEE, 2016.
- [4] E. N. Abdullah, S. Ahmad, M. Ismail, and N. M. Diah, “Evaluating e-commerce website content management system in assisting usability issues,” in *2021 IEEE Symposium on Industrial Electronics & Applications (ISIEA)*, IEEE, 2021.
- [5] M. B. Savadatti and S. K. BV, “Implementation of collaborative filtering in e-commerce recommender systems: An elementary review,” in *2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)*, IEEE, 2024.